JOHANNES KEPLER
UNIVERSITY LINZ

Simon Grundner
k12136610

Institute for Integrated Circuits and Quantum Computing

@ k12136610@students.jku.at

https://github.com/s-grundner/LVA-EIC-KV

December 29, 2025

Tiny Tapeout - MIDI Polyphonic Synthesizer

Technical Report

KV Integrated Circuits Design - WiSe25

Abstract MIDI Polyphonic Synthesizer on the Tiny Tapeout ASIC

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

Abstract

This project implements a polyphonic audio square wave synthesizer in Verilog, which can be controlled via the Musical Instrument Digital Interface (MIDI) protocol. The project is allocated on the TTSKY25b shuttle of the TinyTapeout project and is to be manufactured under the SkyWater 130 nm process. This report documents functionality as well as implementation and discusses design constraints, imposed by the limited physical assigned space on the shuttle.

Acronyms

ASIC Application Specific Integrated Circuit

DAW Digital Audio Workstation

HDL Hardware Description Language

GM1 General MIDI Level 1

GM2 General MIDI Level 2

MIDI Musical Instrument Digital Interface

PDK Process Development Kit

SPN Scientific Pitch Notation

12ET Twelve Tone Equal Temperament

1. Introduction

MIDI is a versatile digital interface, which packs note expressions into a simple protocol. The note expression often origins from a MIDI-controller (source) and contains information about which key on the pianoroll is affected and how it is affected. A key thing to notice is, that the note expression itself does not contain any information about how the audiosignal sounds. The synthesis of the waveform is handled by a MIDI-instrument (sink) and translates the note expression into a audio timesignal. Figure 1 shows how a MIDI-keyboard might interface with a MIDI compatible synthesizer.

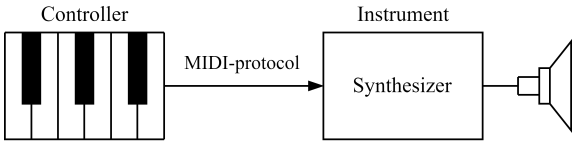

Figure 1: Setup of a controller-instrument-pair communicating via the MIDI-protocol.

The project implements such a MIDI-instrument and converts incoming MIDI-signals into audible squarewaves. The project is written in the Hardware Description Language (HDL) Verilog specifically to be built as a digital Application Specific Integrated Circuit (ASIC) design by the open source LibreLane toolchain. The resulting chip will be taped out in collaboration with the Tiny-Tapeout project.

The Tiny-Tapeout project makes manufacturing ASIC-designs for individuals feasable by combining multiple smaller designs on a single chip under the usage of an open source Process Development Kit (PDK). The chip space is sectioned in tiles, where each tile has around $160 \times 100\,\mu m$ of available space. The project on hand occupies one tile. The active run at this time, TTSKY25b, manufactures the chip under SkyWater 130 nm PDK ruleset.

2. Digital Instruments and Music Theory

Before discussing the design, a few fundamentals of music theory have to be considered and how implementing a digital instrument in silicon is sensible. The MIDI Manufactures Association defines the General MIDI Level 1 (GM1) and its extentsion the General MIDI Level 2 (GM2) standard to provide guidelines for developing MIDI compatible instruments.

2.1 The MIDI-Protocol

The MIDI-protocol is based on a serial asynchronous data transmission, where at most three bytes are sent in succession with a dataformat as shown in Table 1. It is noted, that not all commands require a second data byte which is why it is specified in brackets as an optional byte in Table 1. In this report all required commands provide a second data byte, therefor a length of three bytes will be assumed from now on.

The first byte combines two four-bit values representing a command and a channel number. Depending on the command code, the following databytes can be interpreted accordingly

Table 1: Dataformat of a MIDI-protocol transmission

Byte 1		Byte 2	Byte 3
Statusbyte		Databyte 1	[Databyte 2]
Command	Channel		

Table 2: Excerpt from the MIDI 1.0 specification message summary for channel voice messages [3].
$N = nnnn \in [0-15]$ maps to a MIDI channel number 1-16.
$K = kkkkkkk \in [0-127]$ is the note number. (e.g. which key on a piano is pressed)
$V = vvvvvvv \in [0-127]$ is the velocity (how fast a note is released)

Status	Databytes	Description
1000nnnn	0kkkkkkk 0vvvvvvv	Note Off event. This message is sent when a note is released (ended).
1001nnnn	0kkkkkkk 0vvvvvvv	Note On event. This message is sent when a note is pressed (started).

by a MIDI-sink listening on a pre-set channel. The most important and only command codes used in this project are listed in Table 2.

A seven bit note number (denoted as kkkkkkk in Table 2) correlates per default with the piano keys of a Twelve Tone Equal Temperament (12ET), where the middle C (C_4) is equivalent to a MIDI note number of 60 in decimal [2, sec 2.7.1.1].

2.2 Instrument Tuning

A musical octave is an interval of two notes, where the frequency of the higher note is exactly twice as high as the lower note. The 12ET is a system which divides this octave into twelve notes which are referred to in Scientific Pitch Notation (SPN) as letters from A to G with accidentals (♭, ♯) and an octave number n as shown in Table 3.

Table 3: SPN Nomenclature for notes in a 12 tone tempered octave, where n is the octave-number.

$$B_{n-1} \;\Big|\; C_n \quad \frac{C\sharp_n}{D\flat_n} \quad D_n \quad \frac{D\sharp_n}{E\flat_n} \quad E_n \quad F_n \quad \frac{F\sharp_n}{G\flat_n} \quad G_n \quad \frac{G\sharp_n}{A\flat_n} \quad A_n \quad \frac{A\sharp_n}{B\flat_n} \quad B_n \;\Big|\; C_{n+1}$$

These notes are equally tempered, meaning the frequency-ratio of two adjacent notes is constant for all musical notes. For an octave of 12 notes, this ratio equates to $\sqrt[12]{2} \approx 1.05946$, leading naturally to a logarithmic scale.

To assert actual frequency values to the notes, a pitch standard has to be decided on. GM2 suggests the ISO-16 pitch standard, where note number 69 in decimal (A_4) is tuned to 440 Hz [2, sec 2.7.1.1]. With this reference frequency, the absolute frequency of all notes can be calculated by multiplying with the defined ratio, yielding Equation 1 for frequency $f(K)$ in dependence of the MIDI note number K.

$$f(K) = 440\,\text{Hz} \cdot (\sqrt[12]{2})^{(K-69)} = 440\,\text{Hz} \cdot 2^{(K-69)/12} \tag{1}$$

2.3 Polyphony

Overlapping Squarewave in Time and Frequency Domain. Goal is to show, why each voice is output on a individual Pin.

2.4 Square-Waveform

Why? No Wavetable and DAC required. Simple Digital Output

Spectrum:

- For infinite Time: Base Frequency and Harmonics

- For Finite Time: Sincs and Spectral Leakage. Simulation in Ableton or Matlab

- Note Durations can be random. because they are dependent on the user input.

2.5 MIDI Physical Layer

3. System Overview

3.1 Pinout

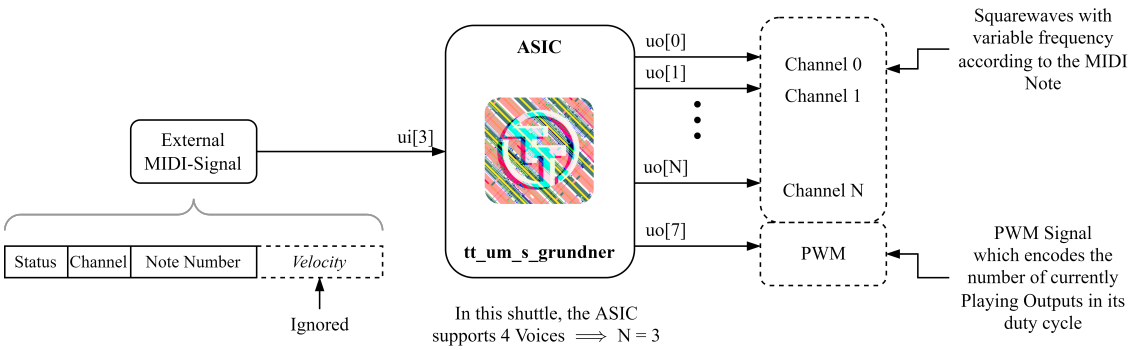

Figure 2: Input and output visualisation and description

3.2 Block Diagram

For this and following sections, backup code with logic diagrams from quartus. Preferrably stick to Abstract Block Diagrams made with tikz

Table 4: Routing statistics

Utilisation / %	Wire length / µm
74.275	23342

Table 5

Category	Cells	Count
Fill	decap fill	733
Combo Logic	a22o a41o and2b a2bb2o o22ai a31o a22oi o21ai a221o o22a o2111ai a211o or4b a21oi a21o o211a and3b a311o o21a o2bb2a o221a a2111o and4b o211ai or4bb a211oi or3b a31oi o41ai o21ba a21bo a32o a221oi o2111a and4bb o32a	228
Tap	tapvpwrvgnd	225
Flip Flops	dfrtp dfstp	191
Multiplexer	mux4 mux2	180
Misc	conb dlymetal6s2s dlygate4sd3	98
AND	and3 and2 and4 a21boi	95
Buffer	clkbuf buf	79
NOR	nor2 xnor2	59
Inverter	inv	49
OR	or2 xor2 or3 or4	48
NAND	nand2 nand3 nand2b	32
Clock	clkinv	10
1069 total cells (excluding fill and tap cells)		

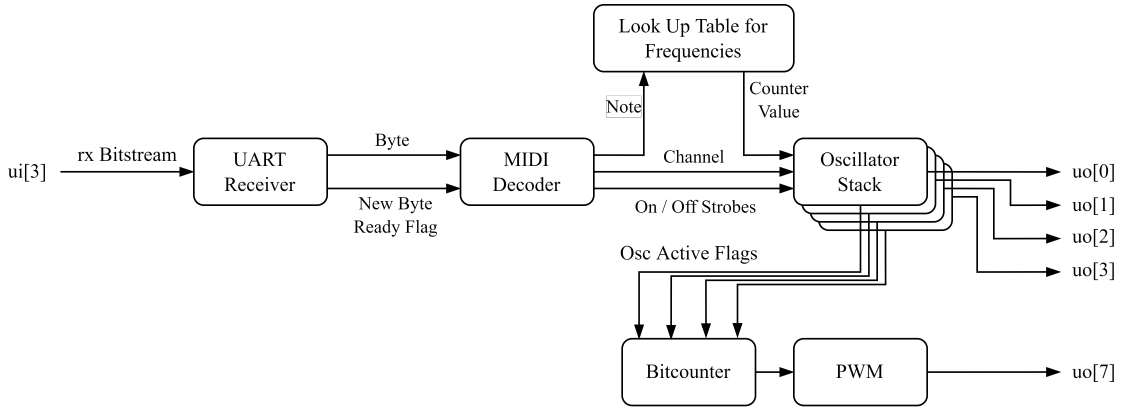

Figure 3

3.3 Layout

4. Reciever and MIDI-Decoder

uart

MIDI Statemachine. has four states. Switches states on new byte ready. Velocity byte is required for the statemachine to switch states correctly, but the value is ignored. A way to interpret velocity, would be to reduce the volume of the output. As the Synthesizer only has one digital amplitude, this is obsolete.

Voice Stealing

MIDI Channel routing

Limitations: According to the General MIDI-System Level 1 (GM-1) Specifications, a Sound Generating Instrument must implement a set of features to ensure compatibility with various controller outputs / midi-sequences [1]. This Project does not comply with GM-1 as even the smallest feature-set requires a much more complex midi statemachine which is beyond the goals of this project.

5. Synthesizer

How is the Frequency Generated?

Count up a defined Number of steps

For each Count, a time interval of 1/fclk elapses.

The Lowest Frequency requires the highest number of counts

The Task is to (space) efficiently determine the Counter Period from the MIDI-Note number

5.1 Frequency Calculation and Generation

Limitations:

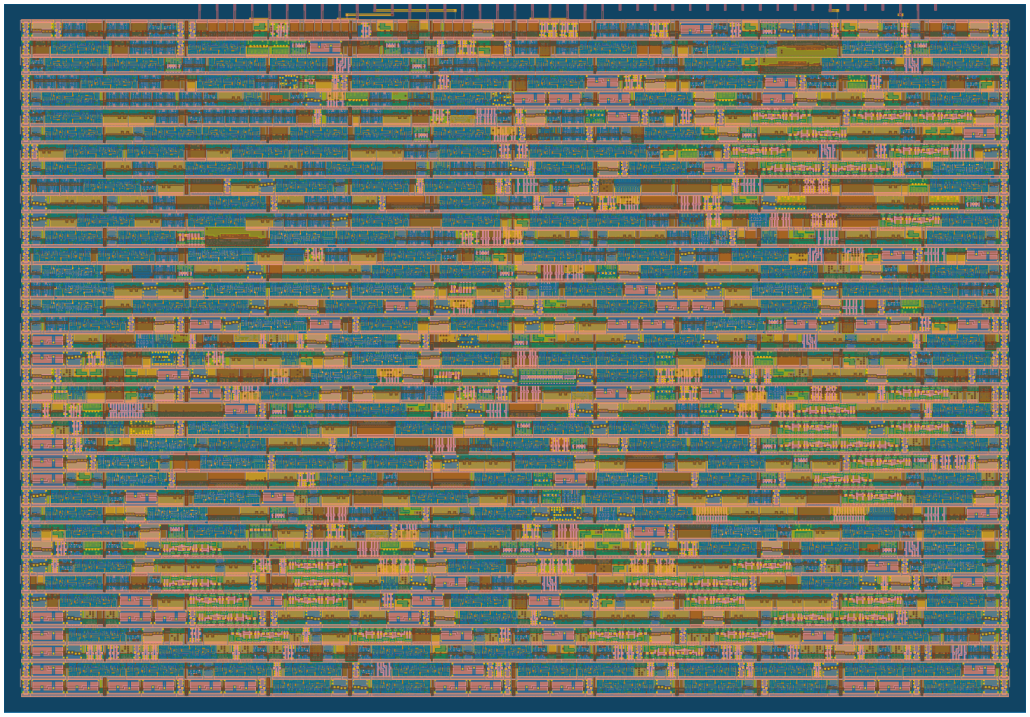

Figure 4: Layout of the Tile

- Frequency Resolution determined by Clock Frequency - Divisions and Modulo operations are Expensive.

Choosing a Clock Frequency

- See reasoning in Py Notebook

Externalize Calculation Logic from the Oscillator Stack

Compare optimisations

- Performing an actual Twelvetone Temparament Calculation on Hardware

- Lookup Table: Save the Countervalues of the Twelve Notes on the Octave to achieve Most precise Counterperiod. What Calculations are required by the oscillators

- Lookup Table: Reduce Bitwitdth and therefor required LUT Size. Compare Size Reduction

Impact of Frequency Deviation

- Systematic error of frequencies from reducing counter resolution. Deviations and calculations from Py-Notebook.

- Spectrum and Harmonics

Are the Deviations Audible?

- Heavily Dependent on the individual Person (Hearing Curve)

- Document Experiment in Ableton Live

5.2 Oscillatorstack

Routing a MIDI a Channel to a fixed Oscillator.

Modularity by a Generating Function: Option to increase number of voices for larger space or future Design optimisations.

6. System Testing

Edge Cases

Limitations. Which tests are omitted. Correctness of the midi Input Bytoe Order for example.

CocoTB Testresults, Design Simulation+validation, Measurements, Waveforms

7. External Hardware

7.1 MIDI-Source

As the device does not fully comply to the GM-1 specs, it is best to provide MIDI inputs from a controlled environment. Pluggin in a full-fledged midi controller directly might lead to unwanted behaviour.

Example of a USB-MIDI controller.

In Principle Direct connection is Possible but a Powersupply is required.

Setting up a Virtual MIDI-Source with a DAW, Hairless MIDI and

PMOD as a PHY for the native Differential midi connection.

7.2 Input Side

Digital Frontend converting simultaneous Notes into individual Channel

7.3 Output Side

Audio Mixing Circuit, Gain Control, Speaker Driver

Option for mild subtractive Sound-Synthesis

- Squarewaves provide all odd harmonics which can be filtered by an external system to create different Sounddesigns

- optimal Would be a Sawtooth as it contains all harmonics but violated the digitality of the ASIC Output.

Table 6: List of used software for development and comissioning this project

Name	Version	Description	Usage
Ableton Live 11 Suite	11.3.43	DAW	

8. Used Software

appendix: Setting up a development env.

References

[1] The MIDI Manufacturers Association. General MIDI System Level 1 Specification. https://midi.org/general-midi-level-1, 1996. Accessed: 2025-12-25.

[2] The MIDI Manufacturers Association. General MIDI System Level 2 Specification. https://midi.org/general-midi-level-2-2, 2007. Accessed: 2025-12-25.

[3] The MIDI Manufacturers Association. Summary of MIDI 1.0 Messages. https://midi.org/summary-of-midi-1-0-messages, 2020. Accessed: 2025-12-25.

Wikipedia References

W MIDI tuning standard - Wikipedia

W Scientific pitch notation - Wikipedia

W 12 equal temperament - Wikipedia

W A440 (pitch standard) - Wikipedia